

Material Maratona de Programação

Murilo Zandoná da Rosa, Lenno Caubi Rhoden, Leonardo Groos Lago

Agosto 2026

Índice

1	Ambiente de Compilação, Depuração e Troubleshooting	3
1.1	Script de Compilação PUC-RJ (Shell Script)	3
1.2	Otimizações do GCC (Compiler Pragmas)	3
1.3	Armadilhas de Hardware e Sinais para Debug Local (feenableexcept)	3
1.4	Checklist Estruturada de Troubleshooting	3
1.4.1	Wrong Answer (WA):	3
1.4.2	Runtime Error (RTE):	3
1.4.3	Time Limit Exceeded (TLE):	3
1.5	Entrada/Saída Rápida (Fast I/O)	3
2	Templates Estruturais, Macros e STL Avançada	3
2.1	Template Estrutural Consolidado (UFMG / PUC-RJ)	3
2.2	Macros de Depuração Inteligentes (TRACE, WATCH, PRINT)	4
2.3	Macetes de STL & Operações Bitwise Velozes	4
2.3.1	Buscas com <code>lower_bound</code> e <code>upper_bound</code>	4
2.3.2	Permutações de Força Bruta	4
2.3.3	Operações de Bits com Built-ins do GCC	4
2.4	Comparação Segura de Ponto Flutuante (Doubles)	4
2.5	Ordered Set (PBDS) e <code>gp_hash_table</code> com Hash Anti-Hack	4
2.5.1	Ordered Set (Conjunto Indexado em $O(\log N)$)	4
2.5.2	<code>gp_hash_table</code> com Hash Anti-Hack	4
3	Estruturas de Dados e Somas Acumuladas	4
3.1	Somas Acumuladas 2D (Prefix Sum 2D)	4
3.2	Disjoint Set Union (DSU / Union-Find)	5
3.3	Segment Tree Clássica (RMQ / RSQ)	5
3.4	Segment Tree Iterativa (Código Enxuto)	5
3.5	Fenwick Tree (Binary Indexed Tree - BIT) 1D	5
3.6	Sparse Table (RMQ Estático em $O(1)$)	5
4	Técnicas de Busca e Resolução de Problemas	6
4.1	Busca Binária no Espaço de Respostas	6
4.1.1	Busca Binária Inteira (Método de Saltos Exponenciais)	6
4.1.2	Busca Binária Contínua (Doubles por Iterações Fixas)	6
4.2	Busca Ternária (Ternary Search)	6
4.3	Técnica dos Dois Ponteiros (Two Pointers)	6
4.4	Janela Deslizante (Sliding Window Minimum/Maximum)	6
5	Algoritmos de Grafos e Árvores	6

5.1	Busca em Profundidade (DFS) e Busca em Largura (BFS) em Grades	6
5.2	Flood Fill (Componentes Conexas em Matrizes)	7
5.3	Menores Caminhos (Dijkstra, Bellman-Ford e Floyd-Warshall)	7
5.3.1	Dijkstra (Arestas Positivas em $O(M \log N)$)	7
5.3.2	Bellman-Ford (Arestas Negativas e Ciclos Negativos em $O(V \cdot E)$)	7
5.3.3	Floyd-Warshall (Todos os Pares em $O(N^3)$)	7
5.4	Árvore Geradora Mínima (Kruskal e Prim)	7
5.4.1	Kruskal (Usa DSU em $O(M \log N)$)	7
5.4.2	Prim (Baseado em Nós em $O(M \log N)$)	7
5.5	Lowest Common Ancestor (LCA) por Binary Lifting	8
5.6	Algoritmo de Mo (Queries Offline em Intervalos)	8
5.7	Ordenação Topológica (Kahn) e Componentes Fortemente Conexas (Tarjan)	8
5.7.1	Ordenação Topológica (Kahn - Fila de Graus de Entrada)	8
5.7.2	Componentes Fortemente Conexas (Tarjan's SCC)	8
6	Programação Dinâmica (DP) Clássica	9
6.1	Maior Soma de Subarray Contíguo (Kadane)	9
6.2	Problema do Troco (Coin Change - Mínimo e Formas)	9
6.2.1	Quantidade Mínima de Moedas para Formar um Valor	9
6.2.2	Número Total de Formas de Formar um Valor	9
6.3	Mochila 0-1 (0-1 Knapsack)	9
6.4	Maior Subsequência Crescente (LIS em $O(N \log N)$)	9
6.5	Maior Subsequência Comum (LCS)	9
6.6	Caminho Mínimo em Grades (Paths in a Grid)	9
6.7	Caixeiro Viajante (TSP) com Bitmask	9
7	Matemática, Teoria dos Números e Convoluções	9
7.1	Aritmética Modular Básica, Euclides Estendido e Inverso Modular Geral	9
7.1.1	MDC, MMC e Exponenciação Rápida com <code>__int128</code> (Sem Overflow)	9
7.1.2	Algoritmo de Euclides Estendido e Inverso Modular Geral	10
7.2	Crivo de Eratóstenes Otimizado (Bitset) e Crivos Lineares	10
7.2.1	Crivo de Eratóstenes Otimizado com <code>bitset</code>	10
7.2.2	Crivo Linear de Funções Multiplicativas em $O(N)$	10
7.3	Multiplicação Rápida de Polinômios por FFT (Floats)	10
7.4	Number Theoretic Transform (NTT) para Inteiros Modulares	10
7.5	Fast Walsh-Hadamard Transform (FWHT - AND/OR/XOR)	11
8	Processamento de Strings	11
8.1	Casamento de Padrões por KMP	11
8.2	Função Z (Z-Algorithm)	11
8.3	Suffix Array e Vetor LCP em $O(N \log N)$	11
9	Geometria Computacional Inteligente	11
9.1	Geometria Inteira Elementar (Struct pt e Vetores - Sem Doubles)	11
9.2	Colinearidade, Pertencimento e Intersecção de Segmentos com Inteiros	12
9.3	Geometria de Pontos Flutuantes (Doubles), Distâncias e Polígonos	12
9.4	Fecho Convexo (Convex Hull - Monotone Chain)	12

1 Ambiente de Compilação, Depuração e Troubleshooting

1.1 Script de Compilação PUC-RJ (Shell Script)

Este script limpa a tela, remove arquivos de saída antigos e compila o código com flags estritos de aviso (`-Wall`), aderência a padrões (`-pedantic`), vinculação de matemática (`-lm`) e informações de debug (`-g`). Se compilar, executa redirecionando o problema.in para a entrada padrão e salva na saída em problema.out

```
#!/bin/sh
clear
rm -f $1.out
if (g++ -o $1 $1.cpp -Wall -pedantic -lm -g) then
    echo "### COMPILOU ###"
    if !(./$1 < $1.in > $1.out) then
        echo "### RUNTIME ERROR ###" >> $1.out
    fi
    less $1.out
fi
```

1.2 Otimizações do GCC (Compiler Pragmas)

Para problemas nos quais a complexidade está próxima do limite e as operações de bit ou laços precisam rodar no limite da CPU do juiz online, use estas diretivas no topo do código

```
#pragma GCC optimize("Ofast")
#pragma GCC optimize("unroll-loops")
#pragma GCC target("avx2")
#pragma GCC target("bmi,bmi2,popcnt,lzcnt")
```

1.3 Armadilhas de Hardware e Sinais para Debug Local (feenableexcept)

O uso de `#pragma GCC optimize("Ofast")` pode esconder falhas graves em ponto flutuante ou gerar comportamentos indefinidos. O código abaixo, inserido sob a diretiva de compilação local, ativa armadilhas no processador para travar o programa instantaneamente no terminal local se houver:

1. Divisão de ponto flutuante por zero.
2. Resultados nulos ou inválidos (NaN).
3. Estouro de limite real (Overflow) [.

Também intercepta o sinal de falha de segmentação (SIGSEGV) ou interrupções abruptas (SIGABRT), forçando uma saída limpa que evita o terminal congelar [.

```
#ifdef LOCAL
#include <fcntl.h>
#include <signal.h>
#endif

void setup_debug_traps() {
#ifdef LOCAL
    // Ativa exceções de ponto flutuante para NaNs (1), divisão
    // por zero (4), e overflow (8) -> 1 + 4 + 8 = 13 (ou 29 para
    // incluir denormais)
    feenableexcept(FE_INVALID | FE_DIVBYZERO | FE_OVERFLOW);

    // Converte falhas de segmentação ou asserções em saídas
    // limpas para encontrar onde travou
    signal(SIGSEGV, [](int) { cerr << "\n[LOCAL_DEBUG] SEGFAULT -
    Acesso invalido de memoria!\n"; _Exit(0); });
    signal(SIGFPE, [](int) { cerr << "\n[LOCAL_DEBUG] SIGFPE -
    Erro aritmetico ou divisao por zero!\n"; _Exit(0); });
    signal(SIGABRT, [](int) { cerr << "\n[LOCAL_DEBUG] SIGABRT -
    Assercao abortada!\n"; _Exit(0); });
#endif
}
```

1.4 Checklist Estruturada de Troubleshooting

Em caso de submissões malsucedidas, siga esta checklist sistematizada na ordem sugerida pelo roteiro do KACTL [:

1.4.1 Wrong Answer (WA):

- ☐ **Limpeza de estruturas:** Você limpou perfeitamente todos os vetores globais, matrizes, mapas e DSU entre múltiplos casos de teste?
- ☐ **Overflow em constantes:** Seus shifts de bit usam literais corretos? (por exemplo, `1LL << 40` ao invés de `1 << 40`, que estoura 32 bits de int).
- ☐ **Variáveis não inicializadas:** Há alguma variável local declarada dentro do `main` ou de funções que não foi explicitamente zerada?
- ☐ **Confusão de variáveis:** Você confundiu N por M , ou usou índices trocados (i por j) nos laços de repetição internos?
- ☐ **Ponto flutuante:** Está comparando doubles diretamente usando `==`? Sempre utilize tolerâncias de imprecisão com EPS.
- ☐ **Módulos de negativos:** Ao calcular o módulo de valores negativos, você garantiu que o resto permaneça positivo? Utilize a fórmula segura `(x % MOD + MOD) % MOD`.

1.4.2 Runtime Error (RTE):

- ☐ **Limites dos arrays:** Seus arrays estáticos têm tamanho suficiente? Garanta uma margem extra segura (ex: `MAXN + 10` ou `MAXN = 100005` para limites de 10^5).
- ☐ **Divisão por zero:** Há qualquer possibilidade de divisão por zero ou resto de divisão (%) por zero em operações dinâmicas?
- ☐ **Pilhas e Recursão:** Estouro de pilha causado por recursão excessivamente profunda e sem podas (branching factor muito grande)

1.4.3 Time Limit Exceeded (TLE):

- ☐ **Passagem por cópia:** Você está passando vetores ou objetos grandes por valor nas assinaturas de funções recursivas (gerando clonagens pesadas)? Altere para referências constantes: `const vector<int>& v`.
- ☐ **Fast I/O esquecido:** Garantiu as linhas de aceleração e substituiu todas as chamadas de `endl` (que força flush lento) por `'\n'?`

1.5 Entrada/Saída Rápida (Fast I/O)

Desvincule os buffers de I/O do C++ e do C clássico no início do `main()`:

```
#define fast_io ios_base::sync_with_stdio(0); cin.tie(0);
// No inicio do main:
// int main() {
//     fast_io
//     setup_debug_traps();
//     ...
// }
```

2 Templates Estruturais, Macros e STL Avançada

2.1 Template Estrutural Consolidado (UFMG / PUC-RJ)

Um template enxuto e veloz contendo abreviações clássicas usadas em treino de programação competitiva :

```
#include <bits/stdc++.h>
using namespace std;

typedef long long ll;
```

```
typedef pair<int, int> ii;
typedef vector<int> vi;
typedef vector<ii> vii;

#define fori(i, n) for (int i = 0; i < (n); ++i)
#define forr(i, a, b) for (int i = (a); i <= (b); ++i)
#define ford(i, a, b) for (int i = (a); i >= (b); --i)
#define sz size()
#define all(x) (x).begin(), (x).end()
#define pb push_back
#define mp make_pair

const int INF = 0x3f3f3f3f;
const ll LINF = 0x3f3f3f3f3f3f3f3fll;
```

2.2 Macros de Depuração Inteligentes (TRACE, WATCH, PRINT)

Para rastrear o valor de variáveis locais sem precisar remover as impressões antes de enviar o código, use macros baseadas em pré-processamento. Para desativar o depurador na submissão, basta alterar a definição para `#define TRACE(x...)` (vazio)

```
// Deixe ativo para debugar localmente, comente/desative na
// submissão
#define TRACE(x...) x
#define PRINT(x...) TRACE printf(x)
#define WATCH(x) TRACE(cout << #x " = " << x << " \n")

// No main da PUC-RJ para desativar buffer e imprimir na hora:
// TRACE(setbuf(stdout, NULL));
```

2.3 Macetes de STL & Operações Bitwise Velozes

2.3.1 Buscas com lower_bound e upper_bound

- **lower_bound**: Retorna iterador para o primeiro elemento maior ou igual ao buscado.
- **upper_bound**: Retorna iterador para o primeiro elemento estritamente maior que o buscado.
- **O Perigo do set**: Nunca use a função genérica `lower_bound(s.begin(), s.end(), x)` em um `set`, pois ela roda em tempo linear $O(N)$. Utilize sempre o método próprio da classe: `s.lower_bound(x)` (que roda em tempo logarítmico $O(\log N)$).

2.3.2 Permutações de Força Bruta

Gere todas as permutações lexicográficas de coleções pequenas ($N \leq 10$) em tempo linear-fatorial $O(N \cdot N!)$. Certifique-se de que a coleção esteja previamente ordenada!

```
vector<int> v = {1, 2, 3};
sort(all(v));
do {
    // Processa permutação atual (ex: v[0], v[1], v[2])
} while (next_permutation(all(v)));
```

2.3.3 Operações de Bits com Built-ins do GCC

Funções nativas extremamente rápidas traduzidas diretamente para instruções Assembly na CPU:

- `__builtin_popcountll(x)`: Retorna o número de bits ativados (bits com valor 1) do tipo `long long`.
- `__builtin_clzll(x)`: Conta quantos zeros existem à esquerda do bit 1 mais significativo. **Atenção**: Comportamento indefinido para $x = 0$.
- `__builtin_ctzll(x)`: Conta quantos zeros existem à direita do bit 1 menos significativo. **Atenção**: Comportamento indefinido para $x = 0$.

2.4 Comparação Segura de Ponto Flutuante (Doubles)

Use tolerância EPS para evitar problemas de arredondamento inerentes à representação binária de ponto flutuante.

```
const double EPS = 1e-9;

int cmpD(double x, double y = 0, double tol = EPS) {
    return (x <= y + tol) ? (x + tol < y) ? -1 : 0 : 1;
}

bool cmp_eq(double x, double y) { return cmpD(x, y) == 0; }
bool cmp_lt(double x, double y) { return cmpD(x, y) < 0; }
```

2.5 Ordered Set (PBDS) e gp_hash_table com Hash Anti-Hack

2.5.1 Ordered Set (Conjunto Indexado em $O(\log N)$)

Adiciona as funcionalidades de buscar por índice (`find_by_order`) e obter a posição por chave (`order_of_key`) no `set` clássico.

```
#include <ext/pb_ds/assoc_container.hpp>
#include <ext/pb_ds/tree_policy.hpp>
using namespace __gnu_pbds;

template<class T>
using ordered_set = tree<T, null_type, less<T>, rb_tree_tag,
    tree_order_statistics_node_update>;

// Uso:
// ordered_set<int> s;
// s.insert(4); s.insert(9);
// *s.find_by_order(0) -> retorna o elemento no índice 0 (valor 4)
// s.order_of_key(9) -> retorna 1 (quantos elementos são
// estritamente menores que 9)
```

2.5.2 gp_hash_table com Hash Anti-Hack

O `unordered_map` comum possui alto overhead de memória e pode ser hackeado por testes de pior caso estruturados para gerar colisões em massa $O(N^2)$. O código abaixo cria um mapa cerca de 3x mais veloz e blindado contra ataques no juiz online.

```
#include <ext/pb_ds/assoc_container.hpp>
using namespace __gnu_pbds;

struct chash {
    const uint64_t C = (1ll)(4e18 * acos(-1)) | 71;
    ll operator()(ll x) const { return __builtin_bswap64(x * C); }
};

// Declaracao do mapa veloz anti-hack:
gp_hash_table<ll, int, chash> safe_map({}, {}, {}, {}, {1 << 16});
```

3 Estruturas de Dados e Somas Acumuladas

3.1 Somas Acumuladas 2D (Prefix Sum 2D)

Permite processar uma matriz de entrada e responder à soma de qualquer subretângulo delimitado por $(r1, c1)$ no canto superior esquerdo e $(r2, c2)$ no canto inferior direito em tempo constante $O(1)$ usando o Princípio de Inclusão-Exclusão.

```
struct PrefixSum2D {
    vector<vector<ll>> p;

    PrefixSum2D(const vector<vector<int>>& grid) {
        int R = grid.size();
        int C = grid[0].size();
        p.assign(R + 1, vector<ll>(C + 1, 0));

        for (int r = 0; r < R; ++r) {
            for (int c = 0; c < C; ++c) {
                p[r + 1][c + 1] = grid[r][c] + p[r][c + 1] + p[r + 1][c] - p[r][c];
            }
        }
    }
};
```

```

    }
}

// Retorna a soma da submatriz do grid original com cantos
// inclusive (0-indexed)
ll query(int r1, int c1, int r2, int c2) {
    if (r1 > r2 || c1 > c2) return 0;
    return p[r2 + 1][c2 + 1] - p[r2 + 1][c1] - p[r1][c2 + 1]
    + p[r1][c1];
}
};

```

3.2 Disjoint Set Union (DSU / Union-Find)

Gerencia de forma extremamente rápida conjuntos disjuntos com união por tamanho/rank e compressão de caminhos.

```

struct UnionFind {
    vector<int> parent, sz;
    UnionFind(int N) {
        parent.assign(N + 1, 0);
        sz.assign(N + 1, 1);
        for (int i = 0; i <= N; ++i) parent[i] = i;
    }
    int find(int i) {
        while (i != parent[i]) {
            parent[i] = parent[parent[i]]; // Compressao simples
            de caminhos
            i = parent[i];
        }
        return i;
    }
    bool isSameSet(int i, int j) {
        return find(i) == find(j);
    }
    void unionSet(int i, int j) {
        if (isSameSet(i, j)) return;
        int x = find(i), y = find(j);
        if (sz[x] < sz[y]) swap(x, y);
        parent[y] = x;
        sz[x] += sz[y];
    }
    int sizeOfSet(int i) {
        return sz[find(i)];
    }
};

```

3.3 Segment Tree Clássica (RMQ / RSQ)

A estrutura recursiva clássica. Permite consultas e atualizações de intervalos gerais em complexidade logarítmica $O(\log N)$.

```

struct SegmentTree {
    int n;
    vector<int> st, A;

    int left(int p) { return p << 1; }
    int right(int p) { return (p << 1) + 1; }

    void build(int p, int L, int R) {
        if (L == R) {
            st[p] = L; // Guarda o índice do menor elemento para
            RMQ
            return;
        }
        int mid = (L + R) / 2;
        build(left(p), L, mid);
        build(right(p), mid + 1, R);
        int p1 = st[left(p)], p2 = st[right(p)];
        st[p] = (A[p1] <= A[p2]) ? p1 : p2;
    }

    int rmq(int p, int L, int R, int i, int j) {
        if (i > R || j < L) return -1;
        if (L >= i && R <= j) return st[p];
        int mid = (L + R) / 2;
        int p1 = rmq(left(p), L, mid, i, j);
        int p2 = rmq(right(p), mid + 1, R, i, j);
        if (p1 == -1) return p2;
        if (p2 == -1) return p1;
        return (A[p1] <= A[p2]) ? p1 : p2;
    }

    void update(int p, int L, int R, int idx, int newVal) {
        if (L == R && L == idx) {
            A[idx] = newVal;
            return;
        }
    }
};

```

```

    int mid = (L + R) / 2;
    if (idx <= mid) update(left(p), L, mid, idx, newVal);
    else update(right(p), mid + 1, R, idx, newVal);
    int p1 = st[left(p)], p2 = st[right(p)];
    st[p] = (A[p1] <= A[p2]) ? p1 : p2;
}

SegmentTree(const vector<int>& _A) {
    A = _A; n = A.size();
    st.assign(4 * n, 0);
    build(1, 0, n - 1);
}

int rmq(int i, int j) { return rmq(1, 0, n - 1, i, j); }
void update(int idx, int newVal) { update(1, 0, n - 1, idx,
    newVal); }
};

```

3.4 Segment Tree Iterativa (Código Enxuto)

A forma iterativa não possui recursão e é incrivelmente rápida de codar na pressa. O intervalo de consulta na função `query` é semi-aberto $[l, r)$.

```

struct IterativeSegTree {
    int n;
    vector<int> tree;

    IterativeSegTree(int sz) {
        n = sz;
        tree.assign(2 * n, 0);
    }

    // Atualiza pontual: pos (0-indexed) recebe o valor
    void update(int pos, int value) {
        for (tree[pos += n] = value; pos > 1; pos >>= 1)
            tree[pos >> 1] = tree[pos] + tree[pos ^ 1]; // Soma
    }

    // Consulta intervalo semi-aberto [l, r)
    int query(int l, int r) {
        int sum = 0;
        for (l += n, r += n; l < r; l >>= 1, r >>= 1) {
            if (l & 1) sum += tree[l++];
            if (r & 1) sum += tree[--r];
        }
        return sum;
    }
};

```

3.5 Fenwick Tree (Binary Indexed Tree - BIT) 1D

Ideal para consultas e atualizações de somas acumuladas dinâmicas em tempo logarítmico de poucas linhas. Usa indexação baseada em 1.

```

struct FenwickTree {
    vector<int> ft;
    FenwickTree(int n) {
        ft.assign(n + 1, 0);
    }
    int rsq(int b) { // Soma acumulada de 1 ate b inclusive (1-indexed)
        int sum = 0;
        for (; b > 0; b -= (b & -b)) sum += ft[b];
        return sum;
    }
    int rsq(int a, int b) { // Soma do intervalo de a ate b inclusive
        return rsq(b) - rsq(a - 1);
    }
    void update(int i, int v) { // Soma v no elemento i (1-indexed)
        for (; i < ft.size(); i += (i & -i)) ft[i] += v;
    }
};

```

3.6 Sparse Table (RMQ Estático em $O(1)$)

Estrutura estática que permite responder consultas RMQ de forma instantânea $O(1)$.

```

struct SparseTable {
    vector<vector<int>>> jmp;
    SparseTable(const vector<int>& V) {
        int n = V.size();
    }
};

```

```

    int levels = 32 - __builtin_clz(n);
    jmp.assign(levels, vector<int>(n));
    for (int i = 0; i < n; i++) jmp[0][i] = V[i];
    for (int k = 1; (1 << k) <= n; k++) {
        int len = 1 << (k - 1);
        for (int j = 0; j + 2 * len <= n; j++) {
            jmp[k][j] = min(jmp[k - 1][j], jmp[k - 1][j + len]);
        }
    }
}

int query(int a, int b) { // Query inclusive [a, b]
    int dep = 31 - __builtin_clz(b - a + 1);
    return min(jmp[dep][a], jmp[dep][b - (1 << dep) + 1]);
}
};

```

4 Técnicas de Busca e Resolução de Problemas

4.1 Busca Binária no Espaço de Respostas

4.1.1 Busca Binária Inteira (Método de Saltos Exponenciais)

Este método de busca binária por saltos evita loops infinitos que ocorrem por arredondamentos na divisão clássica de inteiros $(lo + hi) / 2$. Ele encontra de forma robusta e compacta em 4 linhas o ponto de transição.

```

// Queremos achar o menor valor k que atende a condicao ok(k) = true
// Inicializamos x com o limite inferior (lo - 1)
int x = -1; // Ex: se as respostas validas estao no intervalo [0, max_ans]
for (int b = max_ans; b >= 1; b /= 2) {
    while (!ok(x + b)) x += b;
}
int k = x + 1; // k sera o menor elemento que atende a condicao ok()

```

4.1.2 Busca Binária Contínua (Doubles por Iterações Fixas)

No espaço contínuo, a representação de ponto flutuante gera loops infinitos se usarmos $hi - lo > EPS$. A melhor prática é iterar um número de vezes fixo (ex: 80 ou 100 iterações), o que garante precisão absoluta sem risco de travar.

```

double lo = 0.0, hi = 1e9, mid;
for (int iter = 0; iter < 100; ++iter) { // 100 iteracoes
    alcancam precisao extrema (~1e-30)
    mid = (lo + hi) / 2.0;
    if (can(mid)) {
        hi = mid; // Du lo = mid, dependendo da monotonicidade
    } else {
        lo = mid;
    }
}
double ans = lo;

```

4.2 Busca Ternária (Ternary Search)

Utilizada para encontrar o ponto extremo (mínimo ou máximo) de uma função unimodal côncava ou convexa no espaço contínuo ou discreto.

```

// Exemplo discreto (achar o indice i que maximiza a funcao f(i))
int tern_search_discrete(int lo, int hi) {
    while (hi - lo >= 3) {
        int m1 = lo + (hi - lo) / 3;
        int m2 = hi - (hi - lo) / 3;
        if (f(m1) < f(m2)) lo = m1; // f(m1) < f(m2) -> o maximo
        // esta na direita
        else hi = m2;
    }
    int best_idx = lo;
    for (int i = lo + 1; i <= hi; ++i) {
        if (f(i) > f(best_idx)) best_idx = i;
    }
    return best_idx;
}

```

4.3 Técnica dos Dois Ponteiros (Two Pointers)

Permite realizar varreduras lineares $O(N)$ em arrays ordenados para encontrar ou contar subarrays contíguos com soma específica ou resolver problemas de correspondência como o 2SUM.

```

// Acha se existe dois numeros que somam exatamente x num array ja ordenado
bool solve_2sum(const vector<int>& v, int x) {
    int l = 0, r = v.size() - 1;
    while (l < r) {
        int sum = v[l] + v[r];
        if (sum == x) return true;
        if (sum < x) l++;
        else r--;
    }
    return false;
}

```

4.4 Janela Deslizante (Sliding Window Minimum/Maximum)

Computa o menor (ou maior) valor de todos os subarrays contíguos de tamanho K em complexidade puramente linear $O(N)$ utilizando uma fila monótona (deque) para manter iteradores úteis.

```

vector<int> sliding_window_min(const vector<int>& A, int K) {
    deque<int> dq; // Mantem os indices dos elementos candidatos a minimo
    vector<int> ans;
    for (int i = 0; i < A.size(); ++i) {
        // Remove elementos fora da janela atual
        if (!dq.empty() && dq.front() == i - K) {
            dq.pop_front();
        }
        // Mantem a fila monotonicamente crescente (novos menores
        // removem antigos maiores)
        while (!dq.empty() && A[dq.back()] >= A[i]) {
            dq.pop_back();
        }
        dq.push_back(i);
        // Se a janela ja atingiu o tamanho K, armazena a resposta
        if (i >= K - 1) {
            ans.push_back(A[dq.front()]);
        }
    }
    return ans;
}

```

5 Algoritmos de Grafos e Árvores

5.1 Busca em Profundidade (DFS) e Busca em Largura (BFS) em Grades

Essencial para resolver problemas ad-hoc em tabuleiros bidimensionais.

```

int R, C;
char grid[105][105];
bool vis[105][105];
int dist[105][105];

// Movimentacao em 4 direcoes cardinais (cima, baixo, esquerda, direita)
int dr[] = {-1, 1, 0, 0};
int dc[] = {0, 0, -1, 1};

bool val(int r, int c) {
    return r >= 0 && r < R && c >= 0 && c < C && grid[r][c] != '#';
}

void dfs_grid(int r, int c) {
    vis[r][c] = true;
    fori(i, 4) {
        int nr = r + dr[i], nc = c + dc[i];
        if (val(nr, nc) && !vis[nr][nc]) {
            dfs_grid(nr, nc);
        }
    }
}

void bfs_grid(int sr, int sc) {
    memset(vis, 0, sizeof(vis));
}

```

```
memset(dist, INF, sizeof(dist));
queue<ii> q;

vis[sr][sc] = true;
dist[sr][sc] = 0;
q.push({sr, sc});

while (!q.empty()) {
    auto [r, c] = q.front(); q.pop();
    fori(i, 4) {
        int nr = r + dr[i], nc = c + dc[i];
        if (val(nr, nc) && !vis[nr][nc]) {
            vis[nr][nc] = true;
            dist[nr][nc] = dist[r][c] + 1;
            q.push({nr, nc});
        }
    }
}
}
```

5.2 Flood Fill (Componentes Conexas em Matrizes)

Varre e altera elementos vizinhos colorindo áreas contíguas e contando o tamanho das componentes em tempo linear $O(R \cdot C)$.

```
int floodfill(int r, int c, char cor_original, char nova_cor) {
    if (r < 0 || r >= R || c < 0 || c >= C) return 0;
    if (grid[r][c] != cor_original) return 0;

    grid[r][c] = nova_cor; // Pinta para evitar visitas repetidas
    int ans = 1;
    // Explora as 8 direcoes contiguas (caso do tabuleiro de
    // xadrez ou rei)
    int dr8[] = {1, 1, 0, -1, -1, -1, 0, 1};
    int dc8[] = {0, 1, 1, 1, 0, -1, -1, -1};
    fori(i, 8) {
        ans += floodfill(r + dr8[i], c + dc8[i], cor_original,
            nova_cor);
    }
    return ans;
}
```

5.3 Menores Caminhos (Dijkstra, Bellman-Ford e Floyd-Warshall)

5.3.1 Dijkstra (Arestas Positivas em $O(M \log N)$)

```
vector<ii> adj[MAXN]; // Lista de adjacencia: {vizinho, peso}
int d[MAXN];

void dijkstra(int s, int n) {
    forr(i, 0, n) d[i] = INF;
    d[s] = 0;
    priority_queue<ii, vector<ii>, greater<ii>> pq; // Min-Heap
    // de {distancia, vertice}
    pq.push({0, s});

    while (!pq.empty()) {
        auto [dist, u] = pq.top(); pq.pop();
        if (dist > d[u]) continue;
        for (auto [v, w] : adj[u]) {
            if (d[u] + w < d[v]) {
                d[v] = d[u] + w;
                pq.push({d[v], v});
            }
        }
    }
}
```

5.3.2 Bellman-Ford (Arestas Negativas e Ciclos Negativos em $O(V \cdot E)$)

```
struct Edge {
    int u, v, w;
};
vector<Edge> edges;
int d[MAXN];

bool bellman_ford(int s, int n) {
    forr(i, 0, n) d[i] = INF;
    d[s] = 0;
    fori(i, n - 1) {
        for (auto e : edges) {
```

```
            if (d[e.u] != INF && d[e.u] + e.w < d[e.v]) {
                d[e.v] = d[e.u] + e.w;
            }
        }
    }
    // Detecção de ciclos negativos alcançáveis
    for (auto e : edges) {
        if (d[e.u] != INF && d[e.u] + e.w < d[e.v]) {
            return true; // Existe ciclo de custo negativo!
        }
    }
    return false;
}
```

5.3.3 Floyd-Warshall (Todos os Pares em $O(N^3)$)

```
int dMat[405][405]; // Inicialize a diagonal com 0, arestas
// existentes e INF no restante

void floyd_warshall(int n) {
    fori(k, n) {
        fori(i, n) {
            fori(j, n) {
                if (dMat[i][k] < INF && dMat[k][j] < INF) {
                    dMat[i][j] = min(dMat[i][j], dMat[i][k] +
                        dMat[k][j]);
                }
            }
        }
    }
}
```

5.4 Árvore Geradora Mínima (Kruskal e Prim)

5.4.1 Kruskal (Usa DSU em $O(M \log N)$)

```
struct EdgeK {
    int u, v, w;
    bool operator<(const EdgeK& other) const {
        return w < other.w;
    }
};
vector<EdgeK> edgesK;

int kruskal(int n) {
    sort(all(edgesK));
    UnionFind uf(n);
    int mst_cost = 0, edges_used = 0;
    for (auto e : edgesK) {
        if (!uf.isSameSet(e.u, e.v)) {
            uf.unionSet(e.u, e.v);
            mst_cost += e.w;
            edges_used++;
            if (edges_used == n - 1) break;
        }
    }
    return mst_cost;
}
```

5.4.2 Prim (Baseado em Nós em $O(M \log N)$)

```
vector<ii> adjP[MAXN]; // {vizinho, peso}
bool taken[MAXN];
priority_queue<ii, vector<ii>, greater<ii>> pqP;

void process(int u) {
    taken[u] = true;
    for (auto [v, w] : adjP[u]) {
        if (!taken[v]) pqP.push({w, v});
    }
}

int prim(int s, int n) {
    memset(taken, 0, sizeof(taken));
    process(s);
    int mst_cost = 0, edges_used = 0;
    while (!pqP.empty()) {
        auto [w, u] = pqP.top(); pqP.pop();
        if (!taken[u]) {
            mst_cost += w;
            process(u);
            edges_used++;
            if (edges_used == n - 1) break;
        }
    }
    return mst_cost;
}
```

5.5 Lowest Common Ancestor (LCA) por Binary Lifting

Permite realizar pulos rápidos através de ancestrais de potências de 2, respondendo ao ancestral comum mais próximo entre quaisquer nós estáticos em tempo logarítmico $O(\log N)$.

```
const int LOGN = 20; // ceil(log2(MAXN))
vector<int> gAdj[MAXN];
int memo_parent[LOGN][MAXN];
int l_in[MAXN], l_out[MAXN];
int timer_lca;

void dfs_lca(int u, int p) {
    l_in[u] = ++timer_lca;
    memo_parent[0][u] = p;
    for (int k = 1; k < LOGN; ++k) {
        memo_parent[k][u] = memo_parent[k - 1][memo_parent[k - 1][u]];
    }
    for (int v : gAdj[u]) {
        if (v != p) dfs_lca(v, u);
    }
    l_out[u] = ++timer_lca;
}

bool is_ancestor(int u, int v) { // Retorna true se u eh
    // ancestral de v
    return l_in[u] <= l_in[v] && l_out[u] >= l_out[v];
}

int query_lca(int u, int v) {
    if (is_ancestor(u, v)) return u;
    if (is_ancestor(v, u)) return v;
    for (int k = LOGN - 1; k >= 0; --k) {
        if (!is_ancestor(memo_parent[k][u], v)) {
            u = memo_parent[k][u];
        }
    }
    return memo_parent[0][u];
}

void build_lca(int root, int n) {
    timer_lca = 0;
    dfs_lca(root, root);
}


```

5.6 Algoritmo de Mo (Queries Offline em Intervalos)

Usa ordenação zigzag em blocos de tamanho $\sqrt{N}$ para processar Q consultas de intervalo offline de forma extremamente eficiente. É ideal para contagem de elementos distintos em tempo $O((N + Q)\sqrt{N})$.

```
struct Query {
    int l, r, id;
    int block_idx;
    bool operator<(const Query& other) const {
        if (block_idx != other.block_idx) return block_idx <
            other.block_idx;
        // Ordenacao zigzag para otimizacao de constante do
        // ponteiro da direita
        return (block_idx & 1) ? (r < other.r) : (r > other.r);
    }
};

int block_size;
int cur_ans = 0;
int freq_mo[1000005];
int val_mo[1000005];
int ans_mo[1000005];

void add_mo(int idx) {
    int v = val_mo[idx];
    if (freq_mo[v] == 0) cur_ans++;
    freq_mo[v]++;
}

void remove_mo(int idx) {
    int v = val_mo[idx];
    freq_mo[v]--;
    if (freq_mo[v] == 0) cur_ans--;
}

void solve_mo(vector<Query>& queries, int n) {
    block_size = sqrt(n) + 1;

```

```
for (auto& q : queries) q.block_idx = q.l / block_size;
sort(all(queries));

int cur_l = 0, cur_r = -1;
cur_ans = 0;
memset(freq_mo, 0, sizeof(freq_mo));

for (const auto& q : queries) {
    while (cur_r < q.r) add_mo(++cur_r);
    while (cur_l > q.l) add_mo(--cur_l);
    while (cur_l < q.l) remove_mo(cur_l--);
    while (cur_r > q.r) remove_mo(cur_r--);
    ans_mo[q.id] = cur_ans;
}
}


```

5.7 Ordenação Topológica (Kahn) e Componentes Fortemente Conexas (Tarjan)

5.7.1 Ordenação Topológica (Kahn - Fila de Graus de Entrada)

```
vector<int> adjKahn[MAXN];
int in_deg[MAXN];

vector<int> topo_sort(int n) {
    queue<int> q;
    for (i = 1; i <= n; ++i) {
        if (in_deg[i] == 0) q.push(i);
    }
    vector<int> order;
    while (!q.empty()) {
        int u = q.front(); q.pop();
        order.push_back(u);
        for (int v : adjKahn[u]) {
            in_deg[v]--;
            if (in_deg[v] == 0) q.push(v);
        }
    }
    if (order.size() != n) return {}; // Retorna vazio se houver
    // ciclo!
    return order;
}


```

5.7.2 Componentes Fortemente Conexas (Tarjan's SCC)

```
vector<int> adjTarjan[MAXN];
int dfs_num[MAXN], dfs_low[MAXN], in_stack[MAXN];
vector<int> st_scc;
int timer_tarjan, scc_count;
int comp_id[MAXN];

void dfs_tarjan(int u) {
    dfs_num[u] = dfs_low[u] = ++timer_tarjan;
    st_scc.push_back(u);
    in_stack[u] = 1;

    for (int v : adjTarjan[u]) {
        if (dfs_num[v] == 0) {
            dfs_tarjan(v);
            dfs_low[u] = min(dfs_low[u], dfs_low[v]);
        } else if (in_stack[v]) {
            dfs_low[u] = min(dfs_low[u], dfs_num[v]);
        }
    }

    if (dfs_low[u] == dfs_num[u]) {
        scc_count++;
        while (1) {
            int v = st_scc.back(); st_scc.pop_back();
            in_stack[v] = 0;
            comp_id[v] = u; // Lider da componente
            if (u == v) break;
        }
    }
}

void find_scc(int n) {
    timer_tarjan = scc_count = 0;
    st_scc.clear();
    memset(dfs_num, 0, sizeof(dfs_num));
    memset(in_stack, 0, sizeof(in_stack));
    for (i = 1; i <= n; ++i) {
        if (dfs_num[i] == 0) dfs_tarjan(i);
    }
}


```

6 Programação Dinâmica (DP) Clássica

6.1 Maior Soma de Subarray Contíguo (Kadane)

Computa a subsequência de maior soma contígua em complexidade puramente linear $O(N)$.

```
int kadane(const vector<int>& A) {
    int sum = 0, max_so_far = 0;
    for (int x : A) {
        sum += x;
        max_so_far = max(max_so_far, sum);
        if (sum < 0) sum = 0; // Reinicia se o saldo acumulado
        for negativo
    }
    return max_so_far;
}
```

6.2 Problema do Troco (Coin Change - Mínimo e Formas)

6.2.1 Quantidade Mínima de Moedas para Formar um Valor

```
int get_min_coins(int V, const vector<int>& coins) {
    vector<int> dp(V + 1, INF);
    dp[0] = 0;
    for (int i = 1; i <= V; ++i) {
        for (int c : coins) {
            if (i - c >= 0) dp[i] = min(dp[i], dp[i - c] + 1);
        }
    }
    return dp[V];
}
```

6.2.2 Número Total de Formas de Formar um Valor

```
11 get_total_ways(int V, const vector<int>& coins) {
    vector<ll> dp(V + 1, 0);
    dp[0] = 1;
    for (int c : coins) {
        for (int i = c; i <= V; ++i) {
            dp[i] += dp[i - c];
        }
    }
    return dp[V];
}
```

6.3 Mochila 0-1 (0-1 Knapsack)

Maximiza o valor total sob limite de capacidade de peso W utilizando espaço de memória linear $O(W)$.

```
int knapsack(int W, const vector<int>& weights, const vector<int>
    & values, int n) {
    vector<int> dp(W + 1, 0);
    for (int i = 0; i < n; ++i) {
        for (int w = W; w >= weights[i]; --w) { // Inverso para
            evitar reuso do mesmo item
            dp[w] = max(dp[w], dp[w - weights[i]] + values[i]);
        }
    }
    return dp[W];
}
```

6.4 Maior Subsequência Crescente (LIS em $O(N \log N)$)

Utiliza busca binária em cima do vetor auxiliar de termos finais para resolver em tempo logarítmico acelerado.

```
int lis_nlogn(const vector<int>& a) {
    vector<int> b;
    for (int x : a) {
        auto it = lower_bound(b.begin(), b.end(), x);
        if (it == b.end()) b.push_back(x);
        else *it = x;
    }
    return b.size();
}
```

6.5 Maior Subsequência Comum (LCS)

Computa a maior correspondência entre duas strings em tempo quadrático.

```
int lcs(string s1, string s2) {
    int m = s1.size(), n = s2.size();
    vector<vector<int>> dp(m + 1, vector<int>(n + 1, 0));
    for (int i = 1; i <= m; ++i) {
        for (int j = 1; j <= n; ++j) {
            if (s1[i - 1] == s2[j - 1]) dp[i][j] = dp[i - 1][j - 1] + 1;
            else dp[i][j] = max(dp[i - 1][j], dp[i][j - 1]);
        }
    }
    return dp[m][n];
}
```

6.6 Caminho Mínimo em Grades (Paths in a Grid)

Encontra o custo ótimo deslocando-se apenas para baixo ou para a direita em grades bidimensionais.

```
int max_grid_path(int n, const vector<vector<int>>& grid) {
    vector<vector<int>> dp(n + 1, vector<int>(n + 1, 0));
    for (int y = 1; y <= n; ++y) {
        for (int x = 1; x <= n; ++x) {
            dp[y][x] = max(dp[y][x - 1], dp[y - 1][x]) + grid[y - 1][x - 1];
        }
    }
    return dp[n][n];
}
```

6.7 Caixeiro Viajante (TSP) com Bitmask

Determina o menor ciclo hamiltoniano em complexidade exponencial $O(2^N \cdot N^2)$, adequado para $N \leq 16$.

```
int tsp_memo[16][1 << 16];
int dist_tsp[16][16];
int tsp_nodes;

int tsp(int pos, int mask) {
    if (mask == (1 << tsp_nodes) - 1) return dist_tsp[pos][0]; //
    Retorna ao ponto de partida
    if (tsp_memo[pos][mask] != -1) return tsp_memo[pos][mask];

    int ans = INF;
    for (int nxt = 0; nxt < tsp_nodes; ++nxt) {
        if (!(mask & (1 << nxt))) { // Se não visitada
            ans = min(ans, dist_tsp[pos][nxt] + tsp(nxt, mask |
                (1 << nxt)));
        }
    }
    return tsp_memo[pos][mask] = ans;
}

// Inicializar na main:
// memset(tsp_memo, -1, sizeof(tsp_memo));
// invocar com: tsp(0, 1);
```

7 Matemática, Teoria dos Números e Convoluções

7.1 Aritmética Modular Básica, Euclides Estendido e Inverso Modular Geral

7.1.1 MDC, MMC e Exponenciação Rápida com __int128 (Sem Overflow)

```
11 gcd(11 a, 11 b) { return b ? gcd(b, a % b) : abs(a); }
11 lcm(11 a, 11 b) { return a ? (abs(a) / gcd(a, b)) * abs(b) :
    0; }

11 mod_exp(11 base, 11 exp, 11 m) {
    11 res = 1;
    base %= m;
    while (exp > 0) {
        if (exp & 1) res = (11)((__int128)res * base % m); // Uso
        de __int128 evita overflow intermediário
        base = (11)((__int128)base * base % m);
        exp >>= 1;
    }
}
```

```

    }
    return res;
}

```

7.1.2 Algoritmo de Euclides Estendido e Inverso Modular Geral

Resolve $ax + by = \gcd(a, b)$ e permite computar o inverso modular mesmo quando o módulo não é um número primo.

```

typedef pair<ll, ll> bezout;

bezout find_bezout(ll a, ll b) {
    if (b == 0) return {1, 0};
    auto [x, y] = find_bezout(b, a % b);
    return {y, x - (a / b) * y};
}

ll mod_inverse(ll a, ll m) {
    auto [x, y] = find_bezout(a, m);
    return (x % m + m) % m; // Retorna o menor inverso positivo
}

```

7.2 Crivo de Eratóstenes Otimizado (Bitset) e Crivos Lineares

7.2.1 Crivo de Eratóstenes Otimizado com bitset

```

const int SIEVE_MAX = 10000005; // 10^7
bitset<SIEVE_MAX> is_pr;
vector<int> primes;

void build_sieve() {
    is_pr.set();
    is_pr[0] = is_pr[1] = 0;
    for (int i = 2; i * i < SIEVE_MAX; ++i) {
        if (is_pr[i]) {
            for (int j = i * i; j < SIEVE_MAX; j += i) is_pr[j] = 0;
        }
    }
    for (int i = 2; i < SIEVE_MAX; ++i) {
        if (is_pr[i]) primes.push_back(i);
    }
}

```

7.2.2 Crivo Linear de Funções Multiplicativas em $O(N)$

Computa o valor de funções multiplicativas, como a função de Möbius (μ) e o Totiente de Euler (ϕ), em tempo puramente linear.

```

int phi_linear[MAXN];
int mobius_linear[MAXN];
vector<int> pr_list;
bool is_p_sieve[MAXN];

void build_multiplicative_sieve(int n) {
    memset(is_p_sieve, true, sizeof(is_p_sieve));
    phi_linear[1] = 1;
    mobius_linear[1] = 1;
    is_p_sieve[0] = is_p_sieve[1] = false;

    forr(i, 2, n) {
        if (is_p_sieve[i]) {
            pr_list.push_back(i);
            phi_linear[i] = i - 1;
            mobius_linear[i] = -1;
        }
        for (int p : pr_list) {
            if (i * p > n) break;
            is_p_sieve[i * p] = false;
            if (i % p == 0) {
                phi_linear[i * p] = phi_linear[i] * p;
                mobius_linear[i * p] = 0; // f(p^k) para k >= 2
                na de Mobius eh zero
                break;
            } else {
                phi_linear[i * p] = phi_linear[i] * (p - 1);
                mobius_linear[i * p] = -mobius_linear[i];
            }
        }
    }
}

```

7.3 Multiplicação Rápida de Polinômios por FFT (Floats)

O algoritmo Fast Fourier Transform permite computar a convolução de polinômios de ponto flutuante em complexidade linear-logarítmica $O(N \log N)$.

```

typedef complex<double> base;

void fft(vector<base>& a, bool invert) {
    int n = a.size();
    for (int i = 1, j = 0; i < n; i++) {
        int bit = n >> 1;
        for (; j >= bit; bit >>= 1) j -= bit;
        j += bit;
        if (i < j) swap(a[i], a[j]);
    }
    for (int len = 2; len <= n; len <= 1) {
        double ang = 2 * acos(-1) / len * (invert ? -1 : 1);
        base wlen(cos(ang), sin(ang));
        for (int i = 0; i < n; i += len) {
            base w(1);
            for (int j = 0; j < len / 2; j++) {
                base u = a[i + j], v = a[i + j + len / 2] * w;
                a[i + j] = u + v;
                a[i + j + len / 2] = u - v;
                w *= wlen;
            }
        }
    }
    if (invert) {
        for (base& x : a) x /= n;
    }
}

// Convolucao polinomial real (FFT)
vector<ll> multiply_fft(const vector<ll>& a, const vector<ll>& b) {
    vector<base> fa(a.begin(), a.end()), fb(b.begin(), b.end());
    int n = 1;
    while (n < a.size() + b.size()) n <= 1;
    fa.resize(n); fb.resize(n);
    fft(fa, false); fft(fb, false);
    fori(i, n) fa[i] *= fb[i];
    fft(fa, true);
    vector<ll> result(n);
    fori(i, n) result[i] = round(fa[i].real());
    return result;
}

```

7.4 Number Theoretic Transform (NTT) para Inteiros Modulares

Permite convoluções discretas e exatas sem perda de precisão com floats, operando sob o módulo primo modular 998244353 e raiz primitiva 3.

```

const ll MOD_NTT = 998244353;
const ll G_NTT = 3; // Raiz primitiva

void ntt(vector<ll>& a, bool invert) {
    int n = a.size();
    for (int i = 1, j = 0; i < n; i++) {
        int bit = n >> 1;
        for (; j >= bit; bit >>= 1) j -= bit;
        j += bit;
        if (i < j) swap(a[i], a[j]);
    }
    for (int len = 2; len <= n; len <= 1) {
        ll wlen = mod_exp(G_NTT, (MOD_NTT - 1) / len, MOD_NTT);
        if (invert) wlen = mod_inverse(wlen, MOD_NTT);
        for (int i = 0; i < n; i += len) {
            ll w = 1;
            for (int j = 0; j < len / 2; j++) {
                ll u = a[i + j], v = (a[i + j + len / 2] * w) % MOD_NTT;
                a[i + j] = (u + v) % MOD_NTT;
                a[i + j + len / 2] = (u - v + MOD_NTT) % MOD_NTT;
                w = (w * wlen) % MOD_NTT;
            }
        }
    }
    if (invert) {
        ll n_inv = mod_inverse(n, MOD_NTT);
        for (ll& x : a) x = (x * n_inv) % MOD_NTT;
    }
}

```

7.5 Fast Walsh-Hadamard Transform (FWHT - AND/OR/XOR)

Computa convoluções sob operações bitwise binárias, ideal para problemas em que se necessita somar custos sob condições de lógica binária rápida [.

```
// op: '^' (XOR), '|' (OR), '&' (AND)
template<char op>
void fwht(vector<ll>& a, bool invert) {
    int n = a.size();
    for (int len = 1; 2 * len <= n; len <= 1) {
        for (int i = 0; i < n; i += 2 * len) {
            for (int j = 0; j < len; j++) {
                ll u = a[i + j], v = a[i + len + j];
                if (op == '^') {
                    a[i + j] = u + v;
                    a[i + len + j] = u - v;
                } else if (op == '|') {
                    if (!invert) a[i + len + j] = u + v;
                    else a[i + len + j] = v - u;
                } else if (op == '&') {
                    if (!invert) a[i + j] = u + v;
                    else a[i + j] = u - v;
                }
            }
        }
    }
    if (op == '^' && invert) {
        for (ll& x : a) x /= n;
    }
}
```

8 Processamento de Strings

8.1 Casamento de Padrões por KMP

O algoritmo de Knuth-Morris-Pratt computa o vetor de falhas (função prefixo π) e busca todas as ocorrências de um padrão no texto de forma linear $O(N + M)$ sem necessidade de retrocesso .

```
vector<int> compute_pi(string s) {
    int n = s.size();
    vector<int> p(n, 0);
    for (int i = 1, j = 0; i < n; i++) {
        while (j > 0 && s[i] != s[j]) j = p[j - 1];
        if (s[i] == s[j]) j++;
        p[i] = j;
    }
    return p;
}

vector<int> kmp_search(string text, string pattern) {
    vector<int> p = compute_pi(pattern);
    vector<int> matches;
    int n = text.size(), m = pattern.size();
    for (int i = 0, j = 0; i < n; i++) {
        while (j > 0 && text[i] != pattern[j]) j = p[j - 1];
        if (text[i] == pattern[j]) j++;
        if (j == m) {
            matches.push_back(i - m + 1); // Armazena início físico da ocorrência
            j = p[j - 1];
        }
    }
    return matches;
}
```

8.2 Função Z (Z-Algorithm)

Constrói o vetor Z onde $z[i]$ indica o maior prefixo comum compartilhado entre a string original e o seu sufixo que inicia no índice i .

```
vector<int> get_z(string s) {
    int n = s.size();
    vector<int> z(n, 0);
    int l = 0, r = 0;
    for (int i = 1; i < n; i++) {
        if (i <= r) z[i] = min(r - i + 1, z[i - l]);
        while (i + z[i] < n && s[z[i]] == s[i + z[i]]) z[i]++;
        if (i + z[i] - 1 > r) {
            l = i;
            r = i + z[i] - 1;
        }
    }
}
```

```
    }
    return z;
}
```

8.3 Suffix Array e Vetor LCP em $O(N \log N)$

Ordena lexicograficamente todos os sufixos de um texto estático e calcula o vetor de maiores prefixos comuns adjacentes (LCP) pelo algoritmo de Kasai .

```
struct SuffixArray {
    vector<int> sa, lcp;

    SuffixArray(string& s, int lim = 256) {
        int n = s.size() + 1;
        vector<int> x(s.begin(), s.end()), y(n, ws(max(n, lim)));
        x.push_back(0);
        sa = lcp = y;
        iota(sa.begin(), sa.end(), 0);

        for (int j = 0, p = 0; p < n; j = max(1, j * 2), lim = p) {
            p = j;
            iota(y.begin(), y.end(), n - j);
            for (int i = 0; i < n; i++) if (sa[i] >= j) y[p++] = sa[i] - j;
            fill(ws.begin(), ws.end(), 0);
            for (int i = 0; i < n; i++) ws[x[i]]++;
            for (int i = 1; i < lim; i++) ws[i] += ws[i - 1];
            for (int i = n; i--;) sa[--ws[x[i]]] = i;
            swap(x, y);
            p = 1;
            x[sa[0]] = 0;
            for (int i = 1; i < n; i++) {
                int a = sa[i], b = sa[i - 1];
                x[a] = (y[a] == y[b] && y[a + j] == y[b + j]) ? p - 1 : p++;
            }
            for (int i = 0, k = 0; i < n - 1; lcp[x[i++]] = k) {
                for (k && k--, j = sa[x[i] - 1]; s[i + k] == s[j + k]; k++);
            }
            sa.erase(sa.begin()); // Remove sentinela inicial '$'
            lcp.erase(lcp.begin());
        }
    }
};
```

9 Geometria Computacional Inteligente

9.1 Geometria Inteira Elementar (Struct pt e Vetores - Sem Doubles)

A forma mais inteligente de resolver problemas de geometria em Fase 1 é **eliminar o ponto flutuante**. Ao usar tipos inteiros long long para representar pontos, todas as operações de produto escalar, produto vetorial e teste de orientação espacial são exatas e 100% livres de erros de precisão do processador , .

```
struct pt {
    ll x, y;
    pt(ll x = 0, ll y = 0) : x(x), y(y) {}
    pt operator+(pt p) const { return pt(x + p.x, y + p.y); }
    pt operator-(pt p) const { return pt(x - p.x, y - p.y); }
    pt operator*(ll c) const { return pt(x * c, y * c); }

    // Produto Escalar (Dot Product)
    ll operator*(pt p) const { return x * p.x + y * p.y; }

    // Produto Vetorial 2D (Cross Product - Componente Z)
    ll operator%(pt p) const { return x * p.y - y * p.x; }

    bool operator<(pt p) const {
        return (x != p.x) ? (x < p.x) : (y < p.y);
    }

    bool operator==(pt p) const {
        return x == p.x && y == p.y;
    }
};

// 2 * Area Orientada de 3 pontos no plano [Material-PUC-RJ-2007.pdf]
ll sarea2(pt p, pt q, pt r) {
```

```

    return (q - p) % (r - q);
}

// Teste de orientação: CCW (sentido anti-horário) retorna true
bool ccw(pt p, pt q, pt r) {
    return sarea2(p, q, r) > 0;
}

bool colinear(pt p, pt q, pt r) {
    return sarea2(p, q, r) == 0;
}

```

9.2 Colinearidade, Pertencimento e Intersecção de Segmentos com Inteiros

Implementações geométricas exatas usando apenas inteiros

```

// Verifica se o ponto q está sobre o segmento fechado pr [
// Material-PUC-RJ-2007.pdf]
bool between(pt p, pt q, pt r) {
    return colinear(p, q, r) && (p - q) * (r - q) <= 0;
}

// Verifica se os segmentos fechados pq e rs têm algum ponto em
// comum [Material-PUC-RJ-2007.pdf]
bool seg_intersect(pt p, pt q, pt r, pt s) {
    ll A = (q - p) % (r - p);
    ll B = (q - p) % (s - p);
    ll C = (s - r) % (p - r);
    ll D = (s - r) % (q - r);

    // Cruzamentos estritos
    if ((A > 0 && B < 0) || (A < 0 && B > 0)) &&
        ((C > 0 && D < 0) || (C < 0 && D > 0)) return true;

    // Casos degenerados de pontos sobre segmentos
    if (between(p, r, q)) return true;
    if (between(p, s, q)) return true;
    if (between(r, p, s)) return true;
    if (between(r, q, s)) return true;

    return false;
}

```

9.3 Geometria de Pontos Flutuantes (Doubles), Distâncias e Polígonos

```

struct ptD {
    double x, y;
    ptD(double x = 0, double y = 0) : x(x), y(y) {}
    ptD operator+(ptD p) const { return ptD(x + p.x, y + p.y); }
    ptD operator-(ptD p) const { return ptD(x - p.x, y - p.y); }
    ptD operator*(double c) const { return ptD(x * c, y * c); }
    ptD operator/(double c) const { return ptD(x / c, y / c); }
    double operator*(ptD p) const { return x * p.x + y * p.y; }
    double operator%(ptD p) const { return x * p.y - y * p.x; }
    bool operator<(ptD p) const {
        return (fabs(x - p.x) > EPS) ? (x < p.x) : (y < p.y);
    }
};

double dist(ptD p, ptD q) { return hypot(p.y - q.y, p.x - q.x); }

// Menor distância do ponto p à reta definida por a e b [Material-
// UFMG-2011.pdf]
ptD proj(ptD p, ptD a, ptD b) {
    if (a == b) return a;
    ptD ab = b - a, ap = p - a;
    return a + ab * ((ap * ab) / (ab * ab));
}

double distToLine(ptD p, ptD a, ptD b) {
    return dist(p, proj(p, a, b));
}

// Menor distância do ponto p ao segmento de reta fechado ab [
// Material-UFMG-2011.pdf]
double distToSeg(ptD p, ptD a, ptD b) {
    if ((b - a) * (p - a) < 0) return dist(a, p);
    if ((a - b) * (p - b) < 0) return dist(b, p);
    return distToLine(p, a, b);
}

// 1. Área de Polígono Geral por Shoelace [Material-PUC-RJ-2007.
// pdf]
double poly_area(const vector<ptD>& P) {
    double ans = 0;
    int n = P.size();

```

```

    fori(i, n) ans += P[i] % P[(i + 1) % n];
    return fabs(ans) / 2.0;
}

// 2. Teste de Pertencimento: Ponto dentro de Polígono (Crossing
// Numbers) [Material-PUC-RJ-2007.pdf]
bool inPolygon(ptD p, const vector<ptD>& P) {
    int cnt = 0, n = P.size();
    fori(i, n) {
        ptD q = P[(i + 1) % n];
        // Verifica se esta na fronteira com doubles seguros
        ptD a = P[i] - p, b = q - p;
        if (fabs(a % b) < EPS && (a * b) <= 0) return true;

        cnt ^= ((p.y < P[i].y) - (p.y < q.y)) * ((P[i] - p) % (q
        - p)) > 0;
    }
    return cnt;
}

```

9.4 Fecho Convexo (Convex Hull - Monotone Chain)

O algoritmo Monotone Chain de Andrew reconstrói o fecho convexo de um conjunto de pontos de maneira muito robusta e livre de problemas de precisão com pontos colineares em $O(N \log N)$. Funciona perfeitamente com a struct inteira pt.

```

vector<pt> convexHull(vector<pt> pts) {
    if (pts.size() <= 1) return pts;
    sort(all(pts));
    vector<pt> h(pts.size() + 1);
    int s = 0, t = 0;
    for (int it = 2; it--; s = --t, reverse(all(pts))) {
        for (pt p : pts) {
            while (t >= s + 2 && sarea2(h[t - 2], h[t - 1], p) <=
            0) t--;
            h[t++] = p;
        }
    }
    return {h.begin(), h.begin() + t - (t == 2 && h[0] == h[1])};
}

```